

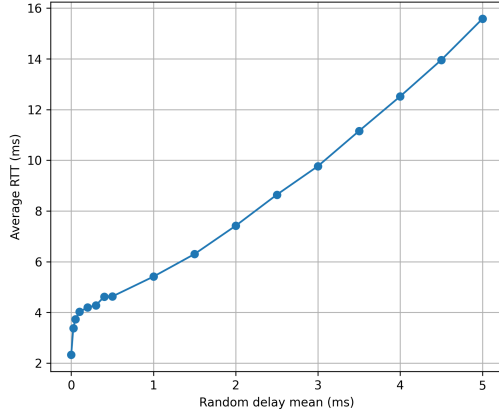
Middle East Technical University
Department of Computer Engineering
Wireless Systems, Networks and Cybersecurity (WINS) Laboratory



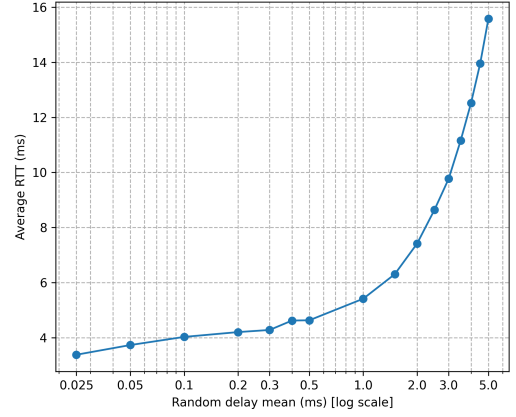
Covert Channel Based On IP Packet Order Phase 1: WARM-UP

CENG519 Network Security
2024-2025 Spring
Term Project Report

Prepared by
Mohamed Eldeeb
Student ID: 2643120
mohamed.eldeeb@metu.edu.tr
Computer Engineering
15 June 2025



(a) Linear scale



(b) Logarithmic scale

Figure 1 . Round-trip time for ping packets with linear and logarithmic scales

1 Introduction

This is the report for the first phase of the term project for Network Security. It explains the relationship between the mean of the random delay added by the processor and the round-trip time (RTT) of the ping packets. Code is available at <https://github.com/mosamadeeb/middlebox/tree/project>.

2 Results and Discussion

2.1 Methodology

The ping command was used to send packets from the secure host to the insecure host. The go-processor was run each time with a different random delay mean, exponentially distributed. A total of 1000 packets were sent at intervals of 10ms for each measurement, and the average RTT was recorded.

2.2 Results

It can be seen from Figure 1 a that the relationship between the exponential mean and the average RTT is linear for higher values, specifically higher than 1ms. Switching to a logarithmic scale as in Figure 1 b shows the relationship better for smaller values. It can be concluded that RTT is more largely affected by the delay when it is higher, and that a smaller delay would probably model real network delay better.

Middle East Technical University
Department of Computer Engineering
Wireless Systems, Networks and Cybersecurity (WINS) Laboratory



Covert Channel Based On IP Packet Order

Phase 2: COVERT CHANNEL

CENG519 Network Security
2024-2025 Spring
Term Project Report

Prepared by
Mohamed Eldeeb
Student ID: 2643120
mohamed.eldeeb@metu.edu.tr
Computer Engineering
15 June 2025

1 Introduction

This is the report for the second phase of the term project for Network Security. A covert channel based on IP packet order was implemented. The implementation is based on the method introduced in [1] with simplified behavior.

Part of the results in this report were remade before the submission of the last phase, as a critical flaw was found in the original methodology.

2 Background and Related Work

2.1 Background

2.1.1 Covert Channel

A covert channel is a form of steganography that relies on an existing communication channel called the overt channel. The purpose of a covert channel is to establish communication in a stealthy manner without disrupting the overt channel's operation. This can be achieved by modulating changes in the properties of the overt communication as data, or by otherwise storing covert information in unused fields in the packets. Examples of covert channels include storage-based channels, which may modify the fields of a packet's header, and time-based channels, which rely on the transmission properties (time between packets, inter-packet jitter, etc.) rather than the data being transmitted.

2.1.2 Packet Order

In the OSI network model, packet arrival order is maintained in connection-oriented networks by the transport layer. TCP, for example, maintains a window for received packets for various reasons, one of which to re-order the packets if they arrive out of order. This is done by checking the sequence number (SEQ) of packets. Thus, if packet reordering occurs in the network before arrival, the application layer of the receiver will still receive the packets in their desired order.

2.2 Related Work

A covert channel design based on packet order is introduced in [1]. The channel exploits the packet reordering phenomenon as a natural occurrence in the Internet. Some of the reasons for packet reordering reviewed in the literature are local parallelism, multi-path forwarding routes, load balancers, and TCP retransmissions. Since the transport layer ensures reception in the order sent, this channel assumes the covert channel receiver will be at the network layer in order to interpret the order of the packets as they arrive.

The channel designed by El-Atawy et al in [1] is based on assigning symbols to the different permutations of a set of reordered packets. Some key parameters are defined as follows:

- k , codeword size, or the number of packets in a permutation,
- l , the number of permutations used to represent the symbols, and
- bpS , the number of bits per symbol.

It should be noted that the inequality $l \leq k!$ has to be satisfied, as $k!$ is the total number of possible permutations of a set of k items.

Symbols represent a binary string of fixed size. The number of bits per symbol (bpS) is equal to $\log_2(l)$. Since it is easier to represent binary data in discrete units, selecting bpS as an integer first would be preferred, and would result in choosing l as 2^{bpS} .

Based on the analysis in [1], the covert channel's capacity can be calculated using the number of packets transmitted in the overt channel (N) as

$$C = bpS \cdot N/k \quad (1)$$

For example, for the set of parameters $k = 4$ and $bpS = 2$, this results in a capacity of 0.5 bits per overt packet.

Another important aspect of the design is the permutation set used for representing the symbols. Some of the suggestions in [1] are choosing the permutations to be uniformly distributed, and to only select even permutations (permutations that need an even number of element swaps starting from the first order). These suggestions allow for advanced capabilities such as error detection and correction.

3 Implementation

The covert sender and receiver are implemented in the middle box environment designed for the Network Security course ([2]). A simplified version of the design made by El-Atawy et al that does not feature error correction was implemented.

The implementation requires the sender and receiver to agree on the parameters (k and bpS) beforehand. The permutations are selected to be evenly spaced to achieve maximum inter-symbol distance. This assists in error detection, and can be leveraged for error correction as in [1], although it is not implemented. One disadvantage of the permutations being uniformly distributed is that it might increase the probability of detection, as orders that are not naturally occurring will appear a lot (e.g., a perfect reverse order: 4, 3, 2, 1). However, for smaller codeword sizes, this effect is not that significant.

4 Results and Discussion

4.1 Methodology

The go-processor implemented in phase 1 of the project was run with a delay mean of 5ms (jitter), in addition to a fixed 20ms delay. The overt data was a randomized stream of bytes with a maximum size of 256 bytes per packet. 3000 packets were sent at intervals of 10ms. The covert data used was the content of the GPLv3 license, although only a small part of it was used as it is too long for the test.

4.2 Results

The results in Figure 1 show the channel capacity (in covert bits delivered per packet) for the performed test over different k and bpS values. The theoretical rates are shown in the figure for comparison.

The capacity results show the channel's ability to deliver covert information with the tested parameters. It can be seen that some configurations are more affected by the jitter than others, which is likely related to the used permutations. Codeword size 4 with 4 bits per

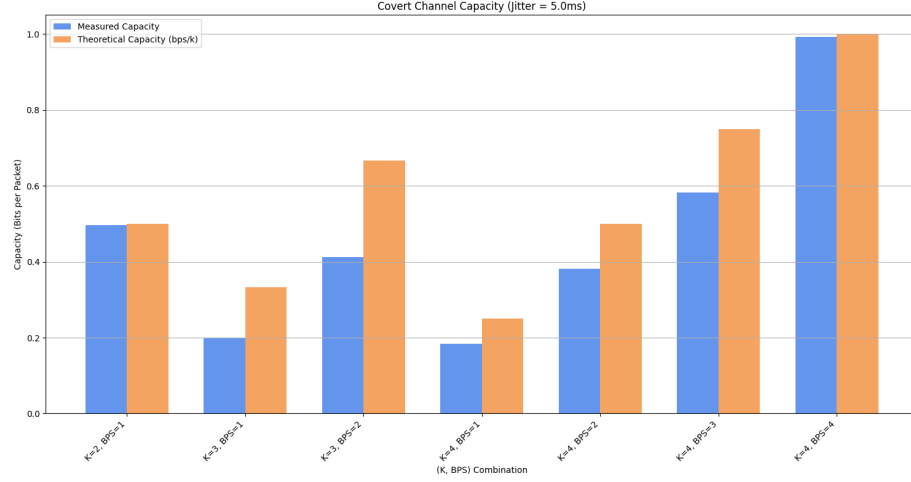


Figure 1 . Channel capacity for different k and bpS values

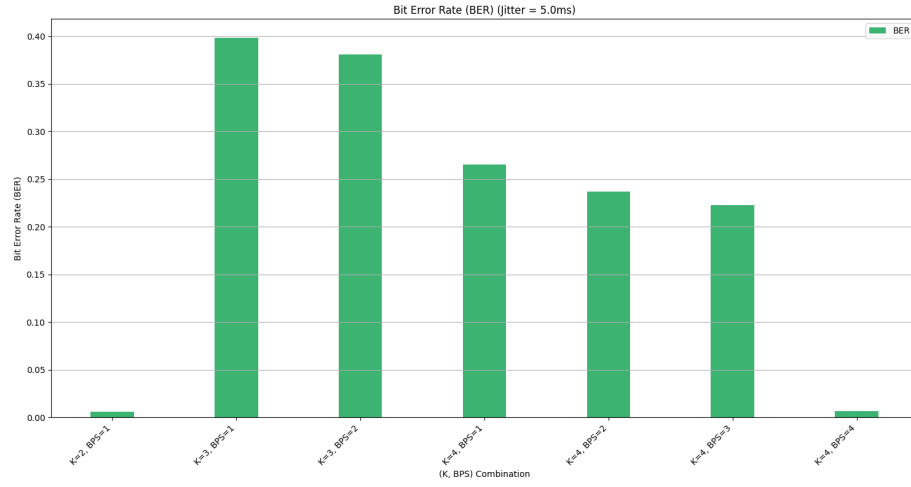


Figure 2 . Bit error rate for different k and bpS values

symbol ($k = 4$, $bpS = 4$) appears to be less affected, reaching a high capacity of 0.993 bits per over packet.

The results in Figure 2 show the bit error rate (BER) for different k and bpS values. These results are follow the same trend as the capacity results, and can give an idea of how the channel results can be used to assess the reliability of the channel.

References

- [1] A. El-Atawy and E. Al-Shaer, “Building Covert Channels over the Packet Reordering Phenomenon,” in *IEEE INFOCOM 2009*, Rio de Janeiro, Brazil: IEEE, Apr. 2009, pp. 2186–2194, ISBN: 978-1-4244-3512-8. DOI: 10.1109/INFCOM.2009.5062143.
- [2] E. Onur, *Middlebox*. [Online]. Available: <https://github.com/cengwins/middlebox>.

Middle East Technical University
Department of Computer Engineering
Wireless Systems, Networks and Cybersecurity (WINS) Laboratory



Covert Channel Based On IP Packet Order Phases 3 & 4: DETECTOR & MITIGATOR

CENG519 Network Security
2024-2025 Spring
Term Project Final Report

Prepared by
Mohamed Eldeeb
Student ID: 2643120
mohamed.eldeeb@metu.edu.tr
Computer Engineering
15 June 2025

Table of Contents

List of Figures	iii
1 Introduction	1
2 Background	1
2.1 Reorder Density	1
3 Detection	1
4 Mitigation	1
5 Results and Discussion	2
5.1 Methodology	2
5.1.1 Detection	2
5.1.2 Mitigation	2
5.2 Results	2
5.2.1 Detection	2
5.2.2 Mitigation	3
6 Conclusion	3

List of Figures

Figure 1	Mitigated channel capacity for different k and bpS values	3
Figure 2	Mitigated bit error rate for different k and bpS values	4

1 Introduction

This is the report for the third and fourth phases of the term project for Network Security. A detector and a mitigator for the covert channel based on IP packet order were implemented.

2 Background

2.1 Reorder Density

RFC 5236 [1] defines several metrics for evaluating packet reordering, including Reorder Density (RD). RD measures the extent of packet reordering by quantifying how far reordered packets have moved from their original sequence. Specifically, it sums the differences between an out-of-order packet's sequence number and the sequence number of the packet that immediately preceded it in the received sequence. Higher RD values indicate more significant reordering. This metric is particularly useful for understanding the magnitude of disorder in a packet stream.

3 Detection

The detector was designed to detect the covert channel by measuring the reorder density of the received packets. The detector is implemented in the go-processor, which processes the packets as they arrive. The implementation uses a buffer to store the sequence numbers of received packets, and calculates the reorder density map in real-time, then logs the results of the detection for each frame of packets.

The detection is based on the following heuristics:

- If the reorder density of the first displacement (-1 or +1) is above 0.15, or
- If the reorder density of the zeroth displacement (i.e. packets that arrived in order) is below the 0.60,

then the packets are considered to be reordered, and the covert channel is detected.

These heuristics were chosen based on the literature reviewed in [2], which suggests that the observed reorder density of first differences is usually below 0.15, and that higher rates of packets arrive in their expected order.

4 Mitigation

The mitigator is designed to effectively prevent covert communication for the IP order based covert channel. While aiming to reduce the channel capacity would be an accepted goal, another approach is used here, which is reordering the packets according to their original order by their sequence numbers. This effectively removes the covert channel, although the resulting channel capacity might not be zero, as the original order may still be used as a valid permutation by the covert channel.

The implementation was made by adding a buffer window in the go-processor, which stores the packets until they are ready to be sent in the correct order. If the window is full, the packets are sent in their sorted order, and the window is cleared. The window size is set to 10 packets to avoid excessive delays, and the packets are sorted by their sequence numbers.

5 Results and Discussion

5.1 Methodology

5.1.1 Detection

The detector was tested by sending packets with a fixed delay of 20ms and a jitter of 5ms. The parameters k and bpS were varied, and the reorder density was measured for different values of these parameters. Additional tests were conducted without the covert channel active in order to evaluate the false positive rate of the detector. The tests were conducted with a total of 5000 packets sent at intervals of 10ms, and each frame consisted of 50 packets.

5.1.2 Mitigation

The mitigator was tested under the same conditions as the capacity tests in Phase 2, with a fixed delay of 20ms and a jitter of 5ms. The parameters k and bpS were varied, and the channel capacity was measured for different values of these parameters. The mitigator was active in the go-processor, providing live mitigation.

5.2 Results

5.2.1 Detection

The detection results are shown in Table 1. The table shows the true positive (TP), false positive (FP), true negative (TN), and false negative (FN) rates for different values of k and bpS . The F1-Score is also calculated for each case, which is a measure of the detector's accuracy. Note that the results are for testing whether a covert channel exists or not, and each row represents the tested configuration.

Since the tests are organized based on covert channel parameters, only the first row is expected to have any true negatives, and the others should not have any false positives (as the covert channel is always active).

The difference in detection rates across the configurations shows the potential for stealthiness of the covert channel. Configurations with lower bits per symbol (i.e. lower effective channel capacity) are less likely to be detected, as they result in less significant displacements. For example, the least detected configuration is $k = 4$ and $bpS = 1$, which has the lowest channel capacity of 0.25 bits per packet, compared to the most detected configuration, $k = 4$ and $bpS = 4$, which has a channel capacity of 1 bit per packet.

Parameters	TP	FP	TN	FN	F1-Score
No covert channel	0	0	1.0	0	N/A
$k=2, bpS=1$	0.77	0	0	0.23	0.8701
$k=3, bpS=1$	0.88	0	0	0.12	0.9312
$k=3, bpS=2$	0.86	0	0	0.14	0.9198
$k=4, bpS=1$	0.66	0	0	0.34	0.7952
$k=4, bpS=2$	0.85	0	0	0.15	0.9198
$k=4, bpS=3$	0.90	0	0	0.10	0.9474
$k=4, bpS=4$	0.91	0	0	0.09	0.9529

Table 1. Detection Results

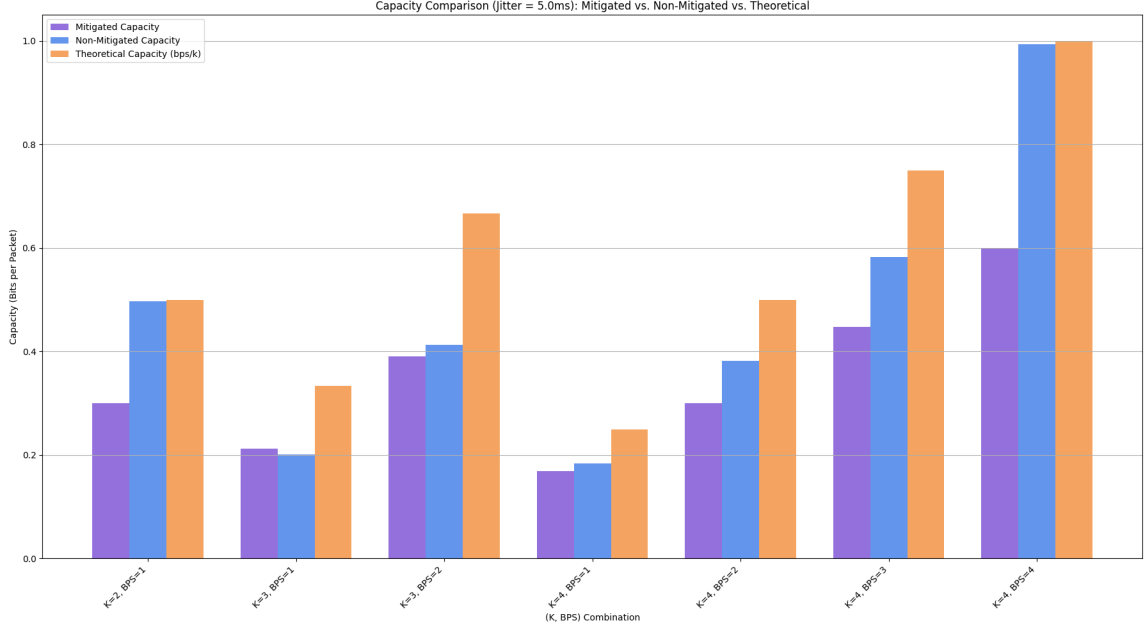


Figure 1 . Mitigated channel capacity for different k and bpS values

5.2.2 Mitigation

Figure 1 shows the mitigated channel capacity for different k and bpS values, along with the non-mitigated and theoretical capacities for comparison, and the bit error rate is shown in Figure 2 .

It can be seen that the mitigator is effective in reducing the channel capacity in general, and is more effective in increasing the bit error rate. For the tested scenarios, the resulting received packet order was always the original order, which prevents the receiver from correctly interpreting the packets as a covert channel.

6 Conclusion

The covert channel based on IP packet order was successfully detected and mitigated. The packet reorder metrics were effective in identifying the covert channel activity. The results demonstrated the effectiveness of the channel in evading detection and the ability of the mitigator to obstruct the covert communication.

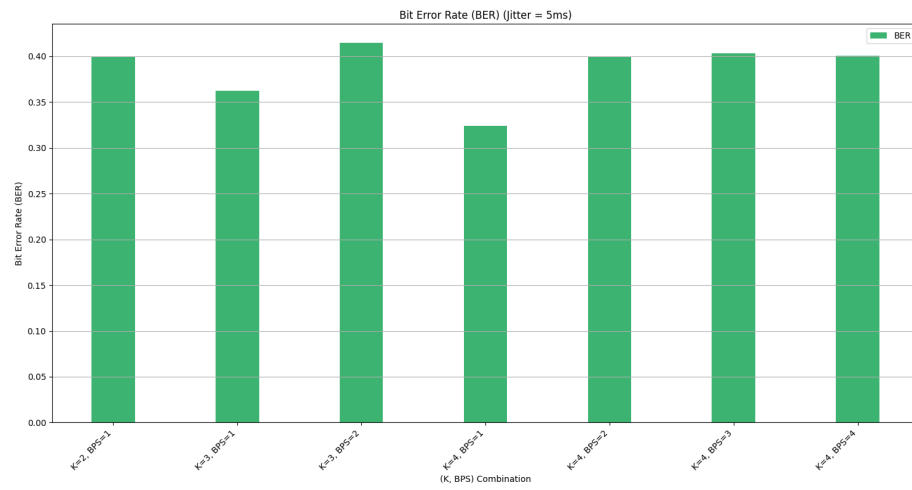


Figure 2 . Mitigated bit error rate for different k and bpS values

References

- [1] *RFC 5236: Improved Packet Reordering Metrics*, Jun. 2008. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5236>.
- [2] A. El-Atawy and E. Al-Shaer, “Building Covert Channels over the Packet Reordering Phenomenon,” in *IEEE INFOCOM 2009*, Rio de Janeiro, Brazil: IEEE, Apr. 2009, pp. 2186–2194, ISBN: 978-1-4244-3512-8. DOI: 10.1109/INFCOM.2009.5062143.